RAIKES SCHOOL

University of Nebraska Lincoln

Decision-Making and Reinforcement Learning

Management Science Principles

Motivation

- Given constraints or a fixed system, we can use optimization to make a decision once.
  - Everything from setting model parameters to supply chain logistics

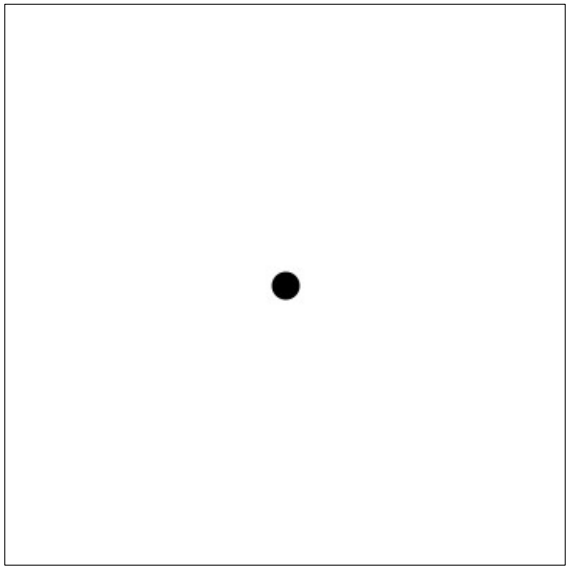
- Evolutionary algorithms included an element of simulation (through generations) and rules (fitness functions) to search an unpredictable space for optimal values.

- Let’s consider simulations as they pertain to making decisions.

Random Walk

Random Walk

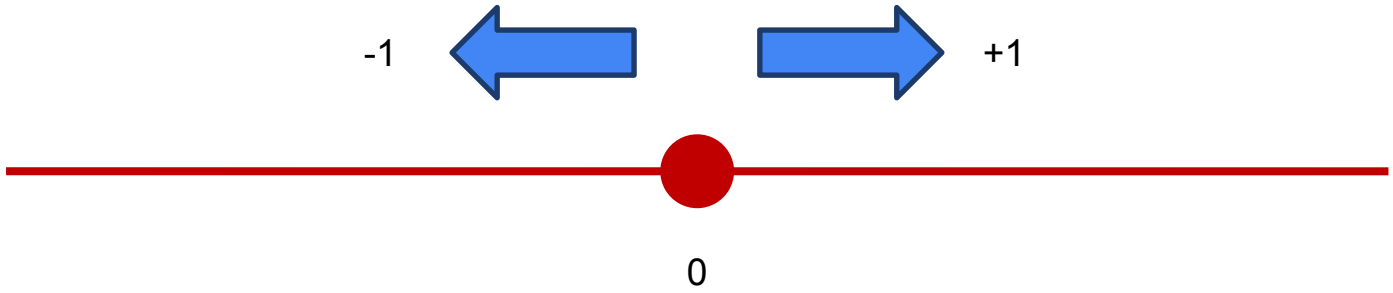
- We have already seen Monte Carlo methods broadly as simulated runs of an experiment to determine outcomes.
- Random walk is one such algorithm.



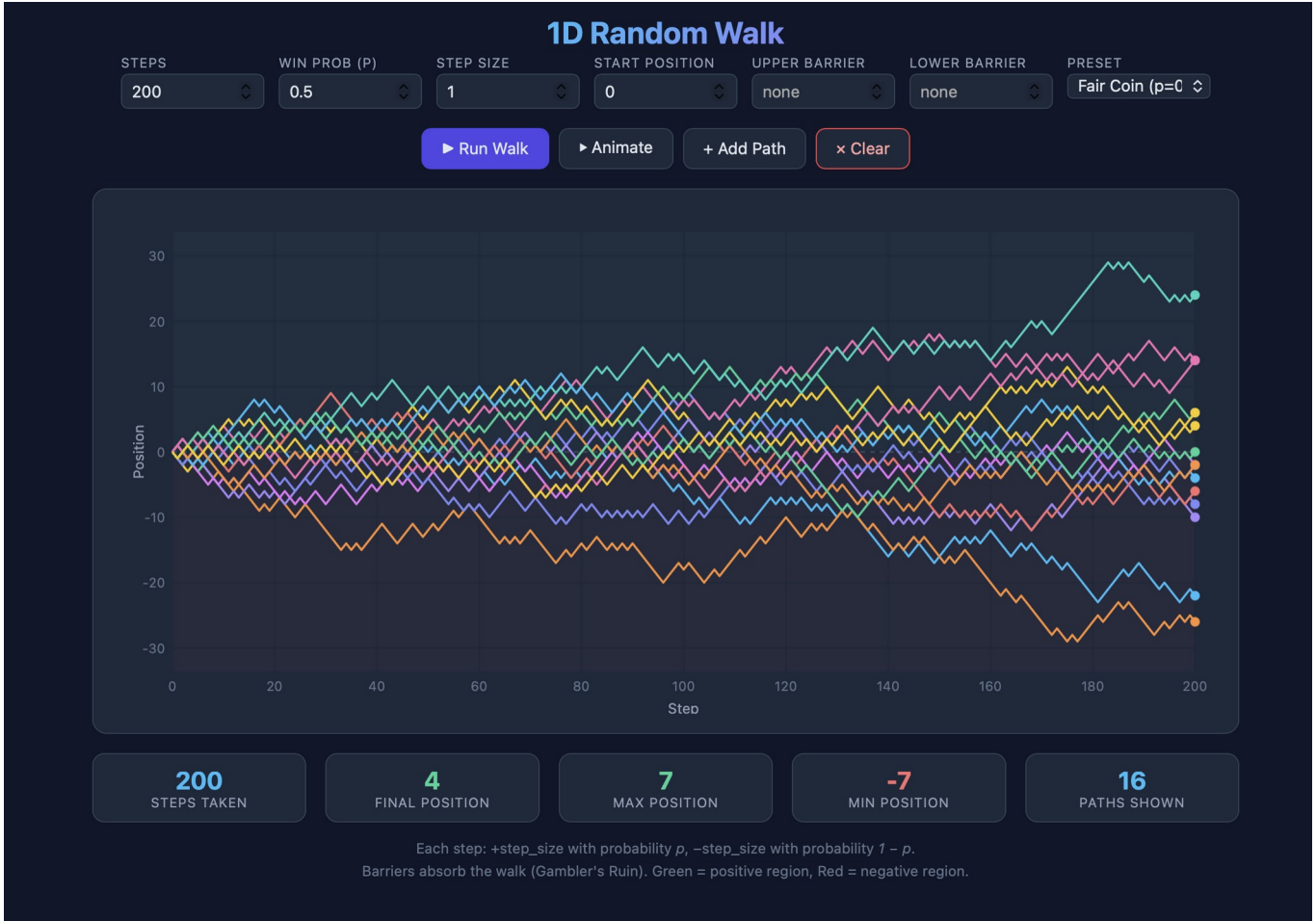
Credit: https://rdlyons.pages.iu.edu/rw/rw.html

1D Random Walk

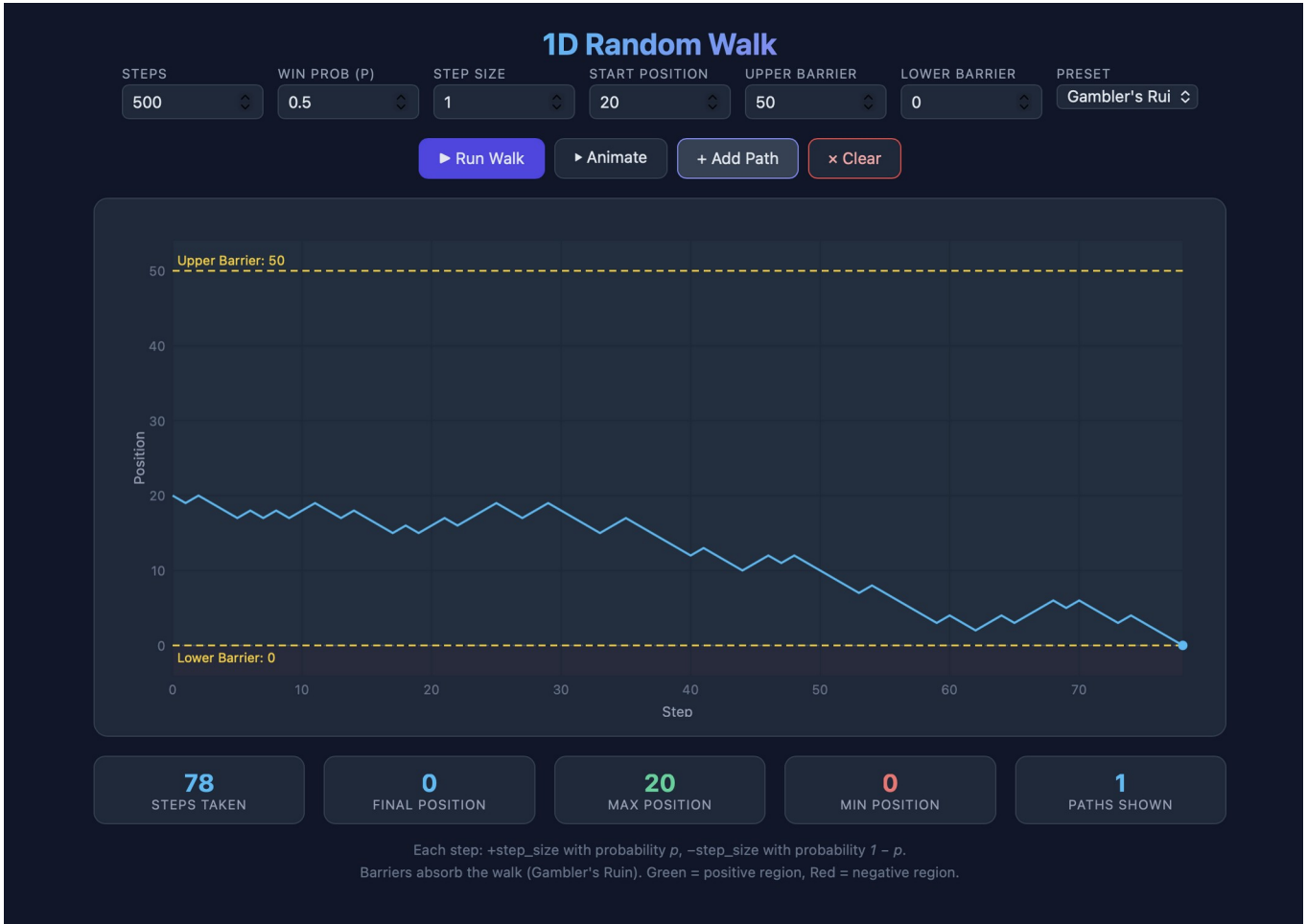
- Consider a scenario where the event can either go left or right: -1 or +1
- We can model this as a 1-dimensional random walk.
- The randomness of the walk direction can be controlled as part of the simulation.



The benefit of simulation is visualizing many sequences or paths

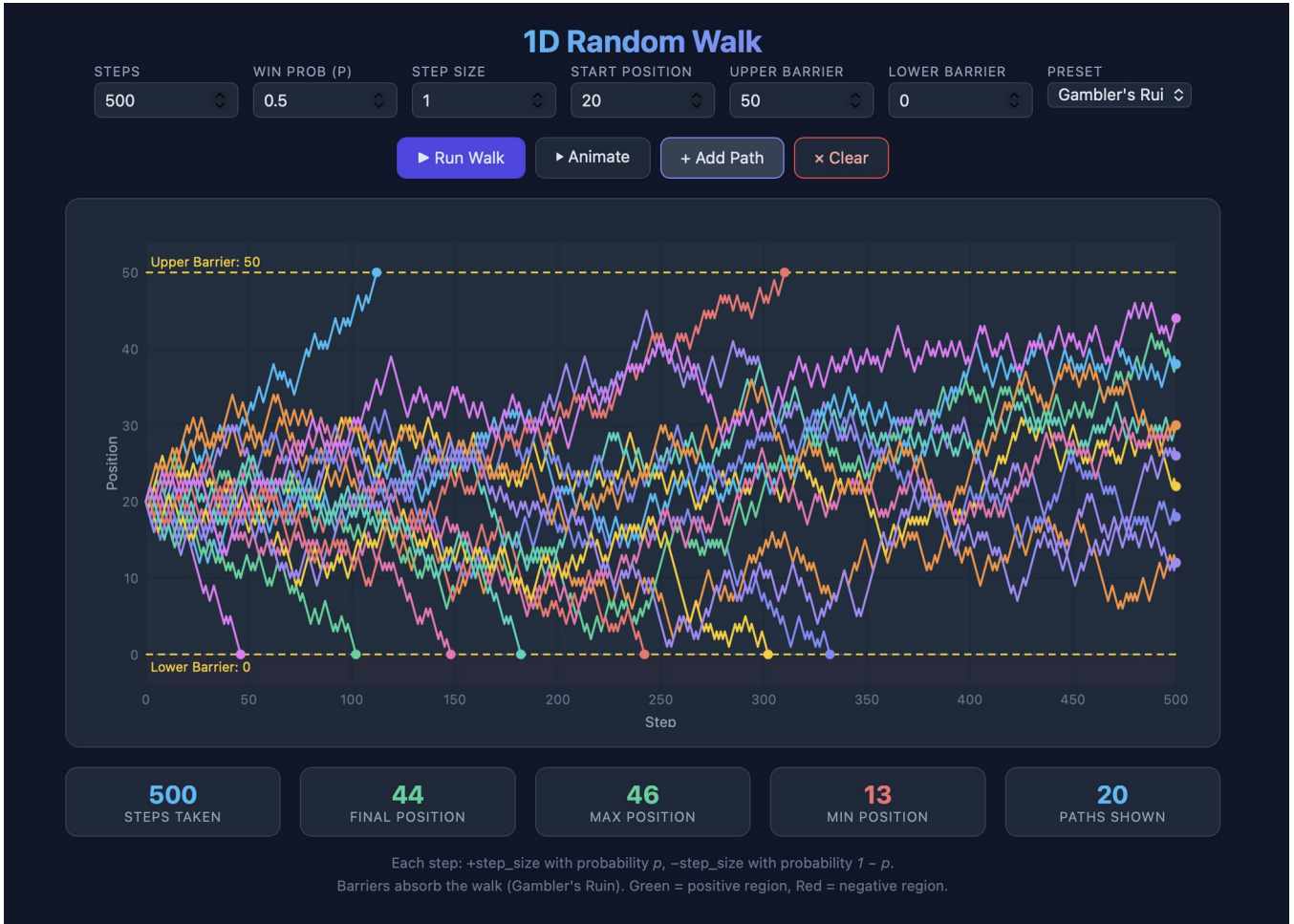


For instance, visualizing minimum thresholds…



End of Slide 7.

… over many iterations



And seeing how even a small change in probability can have major impacts

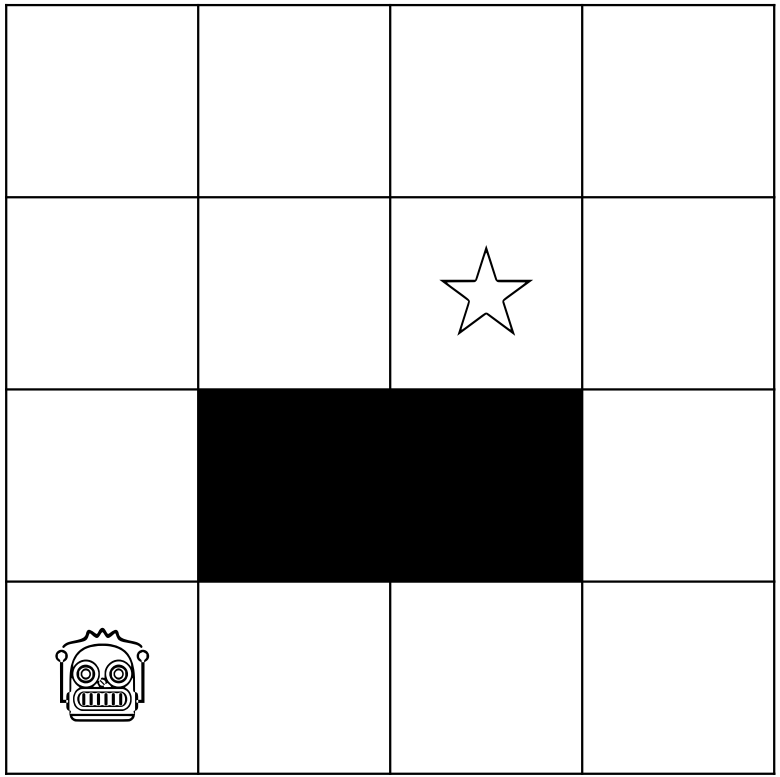
Supporting Dynamicism

- Even simple stochastic systems can produce surprising behavior, and simulations help us see it.
- But… these are still modeling static systems, using a fixed probability across all trials.
  - The only sequential aspect is starting from the point of the last iterations, like a walk where each step is an iteration.
- Consider the gambling scenario: a person may adopt a series of heuristics:
  - If bankroll < $10, bet smaller
  - If on a winning streak, increase bets
  - Quit after 100 rounds
- Such heuristics are common in human decision making with many business parallels:
  - Thresholds at which to reorder inventory
  - Cut ad spend if ROI drops below target
  - Raise prices during peak demand

Let’s try a machine learning approach to selecting rules!

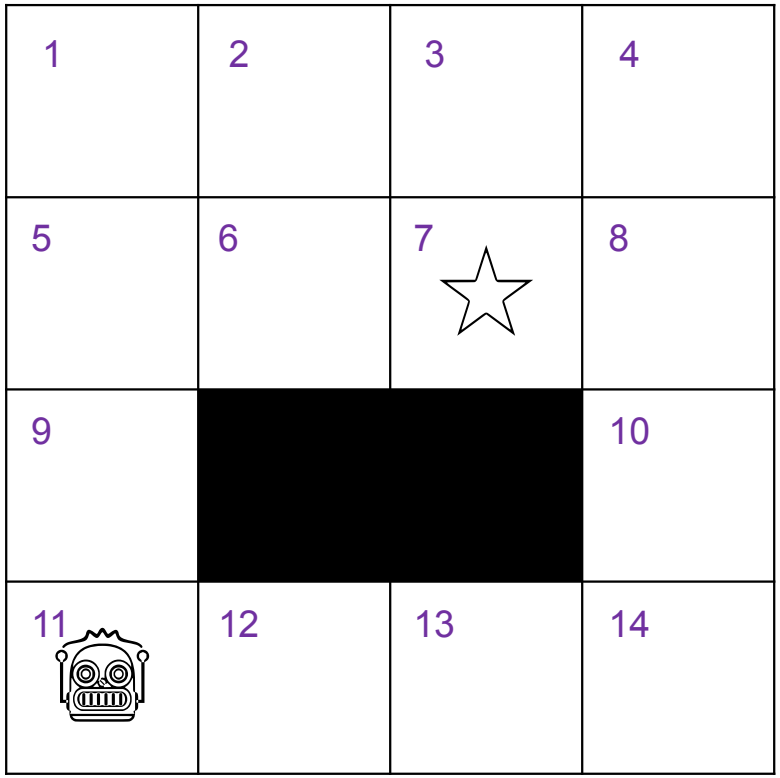
Reinforcement Learning

Reinforcement Learning

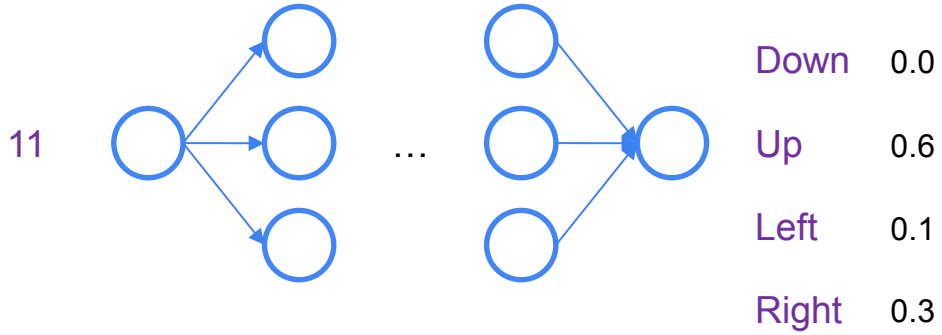


- Broadly, RL models the algorithm as an agent inside the environment.
- The agent should take **actions** that give it a **reward** or move it closer to the goal.
- RL includes **state** as a way to measure action efficacy and seek rewards.

The Policy Network



- Let's construct a simple model to control the actor, call this the Policy or Actor network.
- This network should take as input the current **State** and output an **Action**.
- Here, we could simply encode state as location, but you can imagine more sophisticated state encodings.

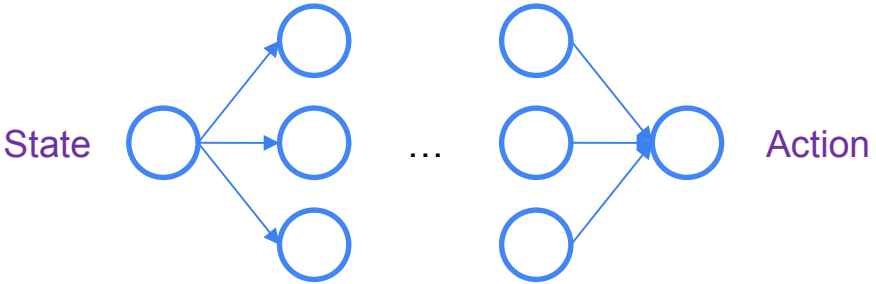


End of Slide 14.

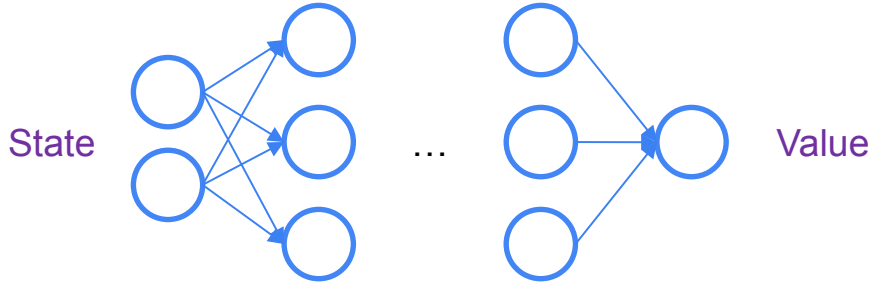
The Value Network

- How can we train the weights for this policy network?
- Modern RL usually creates an adversarial architecture:
  - A Critic or Value network will take an action and state and try to predict how *good* it is.
    - Depending on the technique being used, you may hear this called the value, the fit, the reward, or the “quality” of an action.
      - For instance, Deep Q-Learning (DQL) is concerned with building deep networks that can predict future reward in high-dimensional spaces like entire video game worlds by looking at the pixel values!

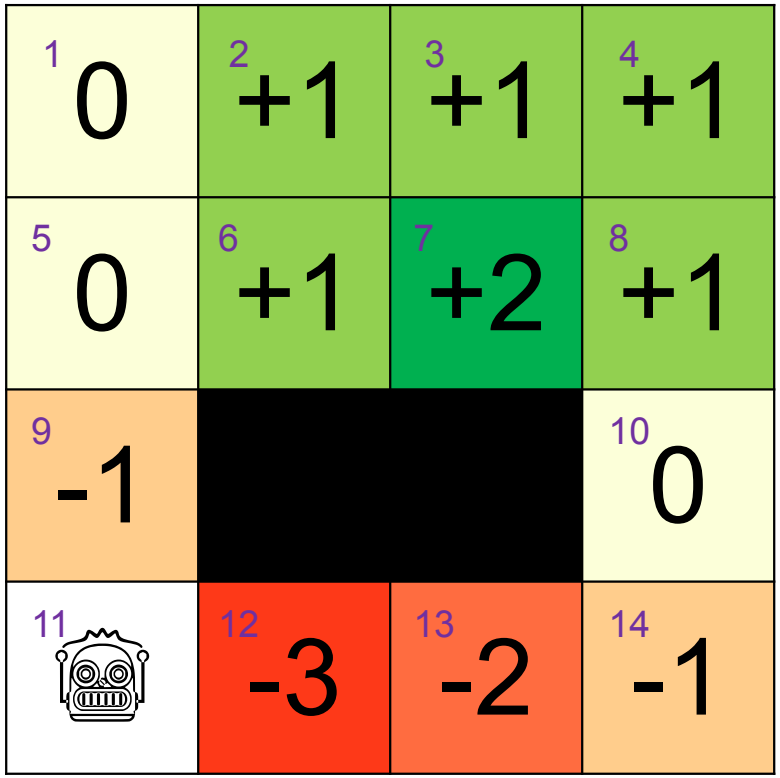
Policy network



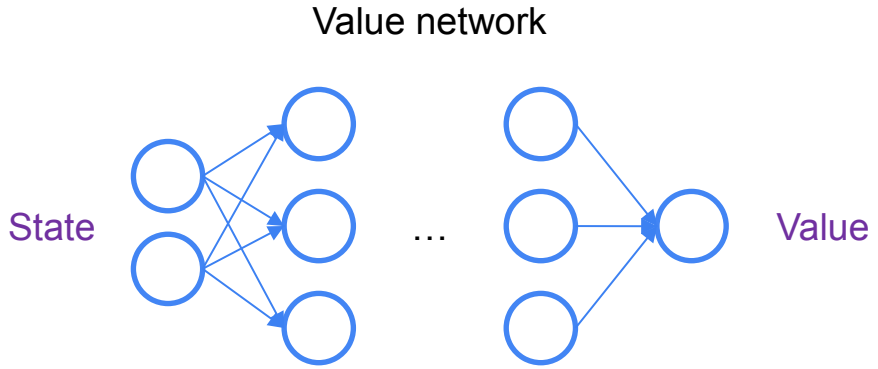
Value network



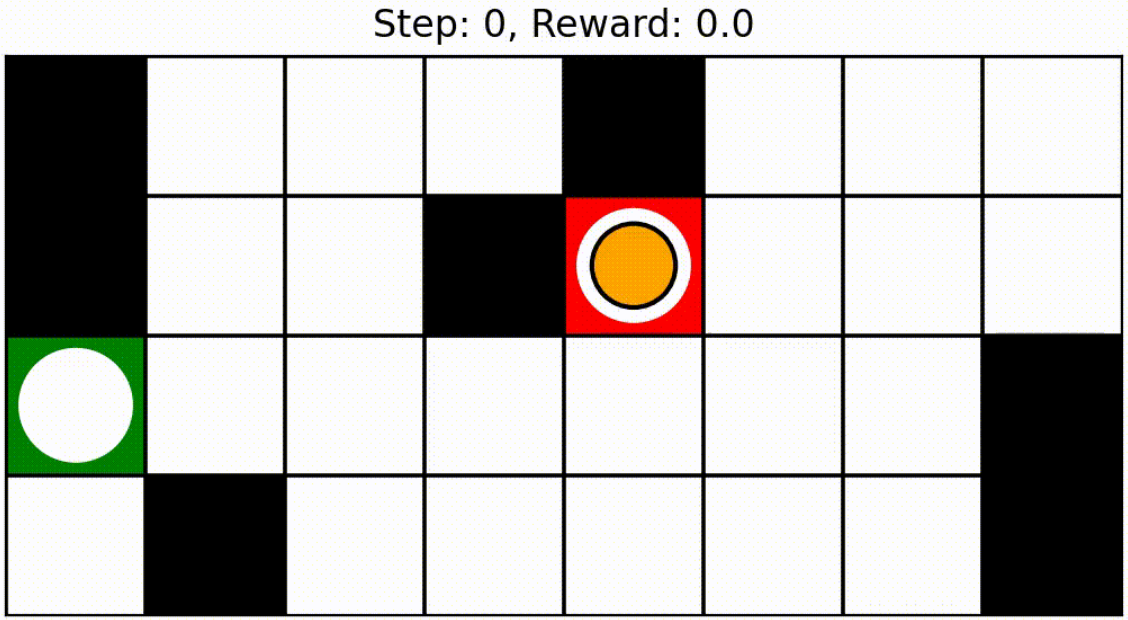
The Value Network



- Let's assign a point value of **2** for our goal and set positions on the map based on those points.
- This is a simplified representative example of what our value network might be trying to predict.



The networks learn optimal actions and values across many iterations.



Credit:
https://medium.com/@leo.damato/building-custom-grid-environments-for-reinforcement-learning-in-gymnasium-a-simple-guide-d433e27424ab

An even simpler RL example

- Consider being at a casino with 3 slot machines and unknown payout rates.
- You get 1,000 pulls.
- What should you do to maximize winnings?

- There is no true state here as the pulls are independent.
- The rewards will be learned by trying.
- Key tradeoff:
  - **Explore**: try different machines to gather information
  - **Exploit**: stick with the machine that seems best at present

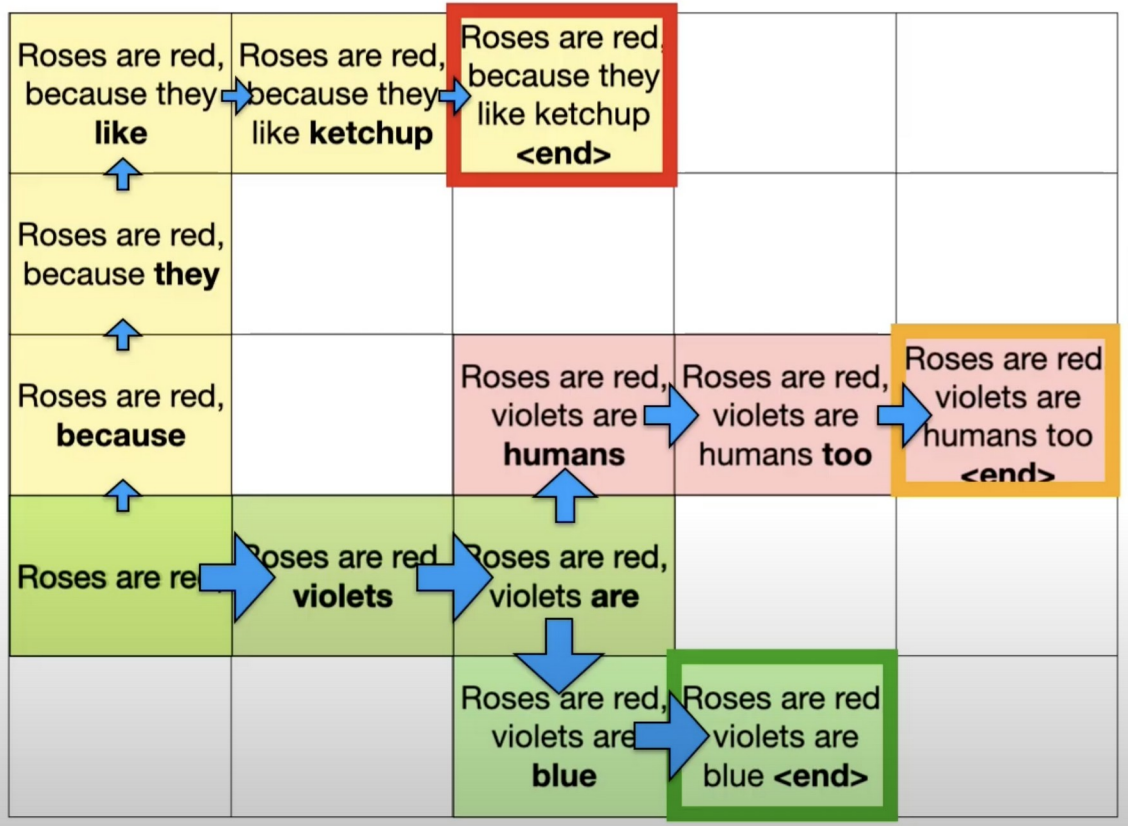
- **Epsilon-greedy strategy**: Pick the best machine most of the time while exploring ε% of the time.

Multi-Armed Bandit

- This is called the ”multi-armed bandit” problem and introduces a very basic form of reinforcement learning where the policy is a blend of exploitation with exploration.

- While it has no state in our construct, it has many business examples that do:
  - A/B testing or ad creative selection can employ exploit vs. explore tradeoffs, but have an effect on interactions and metrics like customer satisfaction.
  - This is why state is significant to reinforcement learning.
    - Relevant to our upcoming discussion of time series data.

Modern RL in Action: Training “human” bots



OpenAI trained a model to mimic human sounding text at scale!

(of course, it’s a high dimensional space, not a grid, but the idea holds)

Credit: Serrano Academy RLHF